

Manual Técnico y Arquitectura del Backend

API REST — Catálogo de Productos

Stack: Python 3.10+ · FastAPI · SQLAlchemy · PostgreSQL

Versión del documento: 1.0

Fecha: Abril 2026

1. Visión General y Arquitectura Monolítica

1.1. Descripción del Sistema

La presente API REST constituye el núcleo de datos de la aplicación Catálogo de Productos. Su responsabilidad es exponer endpoints HTTP que permiten crear, consultar, modificar y eliminar las entidades centrales del dominio: Categorías, Ingredientes, Productos y sus relaciones N:N (ProductoCategoría y ProductoIngrediente). Toda la persistencia se realiza sobre PostgreSQL.

1.2. Decisión Arquitectónica: Monolito en Capas

Se eligió una arquitectura monolítica deliberadamente, priorizando la claridad, la velocidad de desarrollo y la facilidad de mantenimiento. El patrón estructural adoptado es el de Capas Horizontales por Responsabilidad, donde cada directorio del proyecto tiene un rol único e irremplazable en el ciclo de vida de una petición. Ninguna capa puede saltar a otra; la comunicación siempre respeta la jerarquía establecida:

- El Router recibe la petición HTTP y delega al service correspondiente.
- El Service contiene toda la lógica de negocio y coordina la persistencia.
- El UoW (Unit of Work) provee y gestiona la sesión de base de datos.
- El Model representa la estructura de las tablas en PostgreSQL.
- El Schema define los contratos de entrada y salida de datos.

Este enfoque garantiza el Principio de Responsabilidad Única (SRP): si la lógica de una operación cambia, solo se modifica el service; si cambia la estructura de la respuesta, solo se modifica el schema; si cambia la tabla, solo se modifica el model.

1.3. Dominio del Negocio Modelado

El sistema gestiona un catálogo de productos con las siguientes entidades y relaciones:

- Categoría: Entidad de clasificación. Soporta jerarquía propia mediante un campo `parent_id` auto-referencial (FK hacia sí misma), permitiendo categorías padre e hija.
- Ingrediente: Entidad global reutilizable. Representa un componente de producto compartido entre múltiples productos.
- Producto: Entidad central del catálogo. Contiene nombre, descripción, `precio_base`, `stock_cantidad` y disponible.
- ProductoCategoría: Tabla intermedia (clave primaria compuesta) que resuelve la relación N:N entre Producto y Categoría.
- ProductoIngrediente: Tabla intermedia (clave primaria compuesta) que resuelve la relación N:N entre Producto e Ingrediente. Agrega el atributo `es_removible`.

2. Estructura del Proyecto

2.1. Árbol de Directorios

El código fuente se organiza bajo el directorio `backend/app/`, dividido en cinco subdirectorios que corresponden a las cinco capas de la arquitectura:

```
backend/
├── .env
├── requirements.txt
└── app/
    ├── main.py
    ├── database.py
    ├── models/
    │   ├── __init__.py
    │   ├── categoria.py
    │   ├── ingrediente.py
    │   ├── producto.py
    │   ├── producto_categoria.py
    │   └── producto_ingrediente.py
    ├── schemas/
    │   ├── categoria.py
    │   ├── ingrediente.py
    │   └── producto.py
    ├── uow/
    │   └── uow.py
    ├── services/
    │   ├── categoria_service.py
    │   ├── ingrediente_service.py
    │   └── producto_service.py
    └── routers/
        ├── categorias.py
        ├── ingredientes.py
        └── productos.py
```

2.2. `main.py` y `database.py` — Entrada del Sistema

`main.py` es el punto de entrada de la aplicación. Su responsabilidad es: instanciar la aplicación FastAPI, registrar los tres routers (categorias, ingredientes, productos) y ejecutar `crear_tablas()` al iniciar el servidor mediante el mecanismo `lifespan`. Importa explícitamente `app.models` antes de crear las tablas para garantizar que `SQLModel` registre todos los modelos en su metadata interna.

`database.py` encapsula la configuración de conexión con PostgreSQL. Define el motor (`create_engine`) compartido por toda la aplicación, lee la cadena de conexión desde la variable de entorno `DATABASE_URL` definida en el archivo `.env`, y expone la función `crear_tablas()` que llama a `SQLModel.metadata.create_all()`.

2.3. Capa de Modelos — `models/`

Contiene las clases Python que representan directamente las tablas de la base de datos (`SQLModel` con `table=True`). Cada archivo es una entidad del dominio:

- `categoria.py` — Tabla 'categoria'. Columnas: `id` (PK), `parent_id` (FK auto-referencial nullable), `nombre` (VARCHAR 100, UNIQUE), `descripcion` (TEXT nullable), `imagen_url` (TEXT nullable). Relación 1:N hacia `ProductoCategoria`.

-

ingrediente.py — Tabla 'ingrediente'. Columnas: id (PK), nombre (VARCHAR 100, UNIQUE), descripcion (TEXT nullable). Relación 1:N hacia ProductoIngrediente.

- producto.py — Tabla 'producto'. Columnas: id (PK), nombre (VARCHAR 150), descripcion (TEXT nullable), precio_base (FLOAT, >= 0), stock_cantidad (INTEGER, default 0), disponible (BOOLEAN, default true). Dos relaciones 1:N hacia las tablas intermedias, con cascade='all, delete-orphan' para borrado automático de vínculos.
- producto_categoria.py — Tabla intermedia 'producto_categoria'. PK compuesta: producto_id (FK → producto.id) + categoria_id (FK → categoria.id). Relaciones hacia Producto y Categoria.
- producto_ingrediente.py — Tabla intermedia 'producto_ingrediente'. PK compuesta: producto_id + ingrediente_id. Columna adicional: es_removible (BOOLEAN, default false).
- __init__.py — Importa todas las clases de modelo. Es crítico: garantiza que SQLAlchemy registre todos los modelos en metadata antes de ejecutar create_all().

Consideración técnica: Para evitar importaciones circulares entre archivos de modelos, se utiliza 'from __future__ import annotations' combinado con el bloque TYPE_CHECKING. Esto permite declarar los tipos de las relaciones como referencias de string que SQLAlchemy resuelve de forma diferida, sin necesidad de importar las clases en tiempo de ejecución.

2.4. Capa de Schemas — schemas/

Define los contratos de datos de la API usando clases Pydantic puras (sin table=True). Separa completamente la representación de la base de datos de la representación HTTP. Cada módulo contiene múltiples variantes:

- *Base: Campos comunes con sus reglas de validación (min_length, max_length, ge, gt).
- *Create: Hereda de Base. Para ProductoCreate agrega categoria_ids (List[int]) e ingredientes (List[IngredienteAsignacion]) para manejar vínculos N:N desde la creación.
- *Update: Todos los campos son Optional. Permite actualizaciones parciales (PATCH), aplicando cambios solo sobre los campos enviados mediante model_dump(exclude_unset=True).
- *Read: Define la respuesta. Incluye el id generado. Configura model_config = {"from_attributes": True} para leer atributos desde objetos ORM.
- ProductoRead: Incluye las listas anidadas 'categorias' (List[CategoriaResumen]) e 'ingredientes' (List[IngredienteResumen]) que representan los datos relacionados.

2.5. Capa de Unidad de Trabajo — uow/uow.py

Implementa el patrón Unit of Work mediante la clase UnidadDeTrabajo. Su responsabilidad es encapsular el ciclo de vida de la sesión de base de datos, desacoplando completamente a los services del manejo manual de sesiones.

La clase funciona como context manager de Python (protocolo __enter__ / __exit__):

- __enter__: Abre una nueva sesión de SQLAlchemy vinculada al motor de PostgreSQL y la expone como uow.sesion.
- Sin excepciones al salir: __exit__ ejecuta automáticamente commit(), confirmando todos los cambios de la transacción.
- Con excepción al salir: __exit__ ejecuta rollback(), deshaciendo todos los cambios parciales, cierra la sesión y propaga la excepción.

Este comportamiento garantiza atomicidad: una operación que involucra múltiples inserciones (crear un producto con varias categorías e ingredientes) se confirma en su totalidad o no se confirma en absoluto.

2.6. Capa de Servicios — services/

Contiene toda la lógica de negocio. Es la capa de mayor importancia ya que concentra las decisiones del dominio aisladas de cualquier detalle de transporte HTTP.

- `categoria_service.py` — CRUD completo: `crear_categoria`, `obtener_categorias` (con filtro por nombre y paginación), `obtener_categoria`, `actualizar_categoria`, `eliminar_categoria`. El borrado verifica si la categoría tiene productos asociados antes de ejecutar el DELETE; si los tiene, eleva un `ValueError`.
- `ingrediente_service.py` — CRUD completo con la misma protección de integridad referencial en el borrado.
- `producto_service.py` — CRUD completo más `agregar_categoria`, `quitar_categoria`, `agregar_ingrediente`, `quitar_ingrediente`. Contiene el helper interno `_construir_producto_read()` que traduce un objeto ORM (con sus relaciones lazy-cargadas) al schema `ProductoRead` de forma explícita.

Patrón de uso del UoW: cada función abre un bloque `'with UnidadDeTrabajo() as uow:'`. Los schemas de salida se construyen dentro del bloque, mientras la sesión está abierta, para poder acceder a las relaciones mediante lazy loading sin errores de sesión cerrada.

2.7. Capa de Routers — routers/

Los routers son los controladores HTTP. Su responsabilidad es estrictamente de entrada/salida: recibir peticiones, validar parámetros, delegar en los services y devolver respuestas con el código de estado correcto.

- `routers/categorias.py` — Endpoints: POST `/categorias` (201), GET `/categorias` (200, con filtro `?nombre=` y paginación `?skip=&limit=`), GET `/categorias/{id}` (200/404), PATCH `/categorias/{id}` (200/404), DELETE `/categorias/{id}` (204/404/409).
- `routers/ingredientes.py` — Misma estructura CRUD: POST, GET listado, GET detalle, PATCH, DELETE con los mismos códigos de estado.
- `routers/productos.py` — CRUD completo más sub-endpoints N:N: POST `/productos/{id}/categorias` (201), DELETE `/productos/{id}/categorias/{cat_id}` (204/404), POST `/productos/{id}/ingredientes` (201), DELETE `/productos/{id}/ingredientes/{ing_id}` (204/404). El GET listado acepta filtros `?nombre=` y `?disponible=`.

Validaciones con Annotated: el tipo `Annotated` de Python, combinado con `Path()` y `Query()` de FastAPI, declara en un solo lugar las validaciones y metadatos de cada parámetro (rangos, longitudes, descripciones para Swagger). FastAPI los valida automáticamente antes de ejecutar el código del endpoint.

`response_model`: declarado en cada decorador de ruta, actúa como filtro de seguridad. Solo los campos del schema de salida son incluidos en la respuesta; cualquier campo interno del ORM no declarado es descartado silenciosamente.

3. Guía de Configuración y Despliegue Local

3.1. Requisitos Previos

Antes de comenzar, el desarrollador debe tener instalado en su máquina:

- Python 3.10 o superior. Verificable con: `python --version`
- PostgreSQL 14 o superior, en ejecución y accesible localmente en el puerto 5432.
- Cliente de base de datos (recomendado): pgAdmin 4 o DBeaver para inspeccionar las tablas.
- Git para clonar o gestionar el repositorio.
- Editor de código. Se recomienda Visual Studio Code con la extensión de Python.

3.2. Paso a Paso — Configuración del Entorno

Paso 1 — Navegar al directorio del proyecto:

Acceder a la carpeta `backend/` del repositorio. Todos los comandos siguientes deben ejecutarse desde ese directorio.

Paso 2 — Crear el entorno virtual:

Crear un entorno virtual aislado con el módulo estándar `venv` de Python. Se almacena en un directorio `.venv` dentro de `backend/`. Esto evita conflictos entre dependencias de distintos proyectos.

Paso 3 — Activar el entorno virtual:

En Windows: ejecutar `.venv\Scripts\activate`. En macOS/Linux: ejecutar `source .venv/bin/activate`. Al activarse correctamente, el nombre del entorno aparece como prefijo en el prompt de la terminal.

Paso 4 — Instalar las dependencias:

Con el entorno activo, ejecutar: `pip install -r requirements.txt`. Esto instalará: `fastapi[standard]` (framework + `uvicorn`), `sqlmodel` (ORM + validación), `psycopg2-binary` (adaptador PostgreSQL) y `python-dotenv` (carga de variables de entorno).

Paso 5 — Crear la base de datos en PostgreSQL:

Crear una base de datos vacía en el servidor local. El nombre sugerido es `'parcial_db'`. Puede crearse desde pgAdmin, DBeaver o mediante el cliente `psql`.

Paso 6 — Configurar el archivo `.env`:

Editar el archivo `backend/.env` con los datos de conexión del entorno local. La variable `DATABASE_URL` debe seguir el formato estándar de SQLAlchemy: `postgresql://usuario:contraseña@localhost:5432/parcial_db`. Este archivo nunca debe commitearse al repositorio.

Paso 7 — Levantar el servidor en modo desarrollo:

Con el entorno activo, ejecutar: `fastapi dev app/main.py`. El servidor se inicia en `http://127.0.0.1:8000`. Al arrancar, el `lifespan` ejecuta `crear_tablas()`, que crea automáticamente en PostgreSQL todas las tablas definidas en los modelos. Este proceso es idempotente: si las tablas ya existen, no realiza ninguna acción.

4. Flujo de Datos Interno — Ciclo de Vida de una Petición

4.1. Caso de Estudio: POST /productos

Se describe el recorrido completo de la información cuando el sistema recibe una petición de creación de producto. Este flujo involucra todas las capas y representa el escenario más complejo del sistema.

Etapas 1 — Recepción en el Router (routers/productos.py):

FastAPI intercepta la petición POST /productos y la dirige al endpoint `crear_producto()`. El router declara que el cuerpo debe deserializarse con `ProductoCreate` y que la respuesta se formateará con `ProductoRead` y código 201.

Etapas 2 — Validación del Schema de Entrada (schemas/producto.py):

FastAPI entrega el cuerpo JSON a Pydantic para construir una instancia de `ProductoCreate`. Se verifica: que nombre tenga entre 2 y 150 caracteres, que `precio_base` sea ≥ 0 , que `stock_cantidad` sea ≥ 0 , y que `categoria_ids` e ingredientes sean listas válidas. Si cualquier validación falla, Pydantic genera un 422 Unprocessable Entity automáticamente. El código del router nunca se ejecuta ante errores de validación.

Etapas 3 — Delegación al Service (services/producto_service.py):

Con los datos validados, el router invoca `producto_service.crear_producto(datos)`. El router no conoce la base de datos ni la lógica de negocio; solo delega y espera un resultado.

Etapas 4 — Apertura de la Transacción (uow/uow.py):

El service abre un bloque `'with UnidadDeTrabajo() as uow:'`, que llama a `__enter__()` y abre una nueva sesión de `SQLModel` conectada a PostgreSQL. Toda operación dentro del bloque es parte de la misma transacción.

Etapas 5 — Persistencia en múltiples tablas (models/):

El service: (1) Construye una instancia del modelo `Producto` y la agrega a la sesión. (2) Ejecuta `flush()` para que PostgreSQL asigne el id sin confirmar la transacción. (3) Por cada ID en `categoria_ids`, crea un `ProductoCategoria` con la clave compuesta y lo agrega a la sesión. (4) Por cada elemento en ingredientes, crea un `ProductoIngrediente` con `es_removible` y lo agrega a la sesión. (5) Ejecuta un segundo `flush()` para enviar todos los INSERTs. (6) Ejecuta `refresh()` sobre el `Producto` para recargar sus atributos.

Etapas 6 — Construcción del Schema de Respuesta:

Dentro del bloque (sesión abierta), el service invoca `_construir_producto_read(producto)`. Esta función accede a `producto.categorias_link` e `producto.ingredientes_link` mediante lazy loading (SQLAlchemy emite SELECTs automáticamente). Con los datos relacionados disponibles, construye manualmente una instancia de `ProductoRead` con las listas anidadas de `CategoriaResumen` e `IngredienteResumen`.

Etapas 7 — Confirmación de la Transacción:

Al salir del bloque `'with'`, `__exit__()` ejecuta `commit()`. Todos los cambios (producto + vínculos) se confirman atómicamente en PostgreSQL. Si hubiera ocurrido cualquier excepción en los pasos anteriores, `__exit__()` habría ejecutado `rollback()` dejando la base de datos intacta.

Etapas 8 — Serialización y Respuesta HTTP:

El service retorna el objeto `ProductoRead` al router. FastAPI lo serializa a JSON usando el `response_model` declarado en el decorador, que actúa como filtro: solo los campos de `ProductoRead` son incluidos. La respuesta llega al cliente con código HTTP 201 Created y el cuerpo JSON con el producto creado, incluyendo las categorías e ingredientes anidados.

5. Pruebas de la API

5.1. Swagger UI — Documentación Interactiva

FastAPI genera automáticamente una interfaz de documentación interactiva basada en OpenAPI. Con el servidor en ejecución, está disponible en: <http://127.0.0.1:8000/docs>

La interfaz lista todos los endpoints agrupados por tag (Categorías, Ingredientes, Productos), muestra los schemas de entrada y salida de cada operación, e incluye un botón 'Try it out' para ejecutar peticiones reales desde el navegador sin herramientas externas.

También está disponible la documentación en formato ReDoc en: <http://127.0.0.1:8000/redoc>

5.2. Procedimiento de Prueba Recomendado

Seguir el siguiente orden respeta las dependencias entre entidades:

- 1. Crear Categorías: POST /categorias. Verificar respuesta 201 con id generado.
- 2. Crear Ingredientes: POST /ingredientes. Verificar respuesta 201.
- 3. Crear Producto con vínculos: POST /productos con categoria_ids e ingredientes referenciando los IDs anteriores. Verificar que la respuesta 201 incluya las listas anidadas.
- 4. Consultar listado con filtros: GET /productos?nombre=xxx&disponible=true. Verificar filtrado y paginación con skip/limit.
- 5. Actualización parcial: PATCH /productos/{id} enviando solo precio_base. Verificar que los demás campos no cambian.
- 6. Gestión N:N: POST /productos/{id}/ingredientes para agregar, luego DELETE /productos/{id}/ingredientes/{ing_id} para quitar.
- 7. Errores de validación: Crear una categoría con nombre de 1 carácter. Verificar respuesta 422 con detalle de campos.
- 8. Recurso inexistente: GET /categorias/99999. Verificar respuesta 404.
- 9. Conflicto de integridad: DELETE /categorias/{id} con productos asociados. Verificar respuesta 409.
- 10. Borrado en cascada: DELETE /productos/{id}. Verificar en pgAdmin que los registros en producto_categoria y producto_ingrediente fueron eliminados.

5.3. Tabla de Códigos de Estado HTTP

La siguiente tabla resume los códigos de estado que el equipo puede esperar de la API:

Código	Nombre	Cuándo ocurre
200 OK	Éxito	Lectura (GET) o actualización (PATCH) exitosa.
201 Created	Creado	Inserción exitosa de un nuevo recurso (POST).
204 No Content	Sin Contenido	Eliminación exitosa (DELETE). La respuesta no tiene cuerpo.
404 Not Found	No Encontrado	El recurso solicitado no existe en la base de datos.
409 Conflict	Conflicto	Intento de eliminar una entidad con dependencias activas en otras tablas.
422 Unprocessable	Error de Validación	El cuerpo no cumple las reglas del schema. FastAPI lo genera automáticamente.
500 Internal Server	Error Interno	Error inesperado en el servidor. En modo dev se muestra el traceback en terminal.